

# Swift 面试高频五连问：Optional、Task、Actor、Concurrency 和 OC 差异

原鸣清

2026-05-13

👁 154

🕒 阅读12分钟

清

LV.2

开发 @鸣辰

1.5k

阅读

2

粉丝

关注

私信

Swift 这门语言最有意思的地方在于：它表面上写起来很现代、很安全，但背后其实藏了很多编译器、运行时和 ABI 设计。

下面这几个问题，刚好能把 Swift 的类型系统、并发模型、运行时设计和 Objective-C 的差异串起来。

## 1. Optional 的实现：T? 不是语法糖那么简

Swift 里的 Optional，本质上是一个泛型枚举：

▼ java

体验AI代码助手

代码解读

复制代码

```
1 @frozen public enum Optional<Wrapped>: ExpressibleByNilLiteral {
2     case none
3     case some(Wrapped)
4 }
```

也就是说：

▼ javascript

体验AI代码助手

代码解读

复制代码

```
1 var name: String?
```

等价于：

▼ javascript

体验AI代码助手

代码解读

复制代码

```
1 var name: Optional<String>
```

它要么是：

▼ sql

体验AI代码助手

代码解读

复制代码

```
1 .none
```

要么是：

▼ sql

体验AI代码助手

代码解读

复制代码

```
1 .some("Tom")
```

收起 ^

- 实现：T?不是语法糖那...
- t 要这么设计？
- 内存布局也有优化
- 常见语法，本质都是模...
- detached的区别
- 承当前上下文
- ied {}：切断上下文
- Task?
- Task.detached?
- 原理？MainActor 一定

- 定位-ProFile
- 赞
- 币展示：从一个需求到...
- 赞
- onView 高可用架构：复...
- 赞
- SplitViewController配置...
- 赞
- 原理、消息机制、多继承...
- 赞

- 秒无感页面跳转？聊聊被...
- 269阅读 · 6点赞
- 工程深度教程：从 AGEN...
- 102阅读 · 2点赞
- 指南：8 个正在重塑你...
- 112阅读 · 3点赞

Swift 官方文档也强调，Optional 值在许多上下文里必须先 unwrap 才能使用，这是 Swift 类型安全的一部分。

## 为什么 Swift 要这么设计？

Objective-C 里 nil 是一个运行时概念：

ini

体验AI代码助手 代码解读 复制代码

```
1 NSString *name = nil;
```

很多错误只有运行时才暴露，比如给 nil 发消息可能悄悄什么都不做。

Swift 则把“可能为空”提升到了类型系统：

javascript

体验AI代码助手 代码解读 复制代码

```
1 let a: String = "hello"
2 let b: String? = nil
```

String 和 String? 是两个不同类型。你不能直接把 String? 当成 String 用。

swift

体验AI代码助手 代码解读 复制代码

```
1 let name: String? = "Tom"
2
3 // 编译错误
4 print(name.count)
5
6 // 正确
7 if let name {
8     print(name.count)
9 }
```

这就是 Optional 的核心价值：把空值风险从运行时提前到编译期。

## Optional 的内存布局也有优化

理论上，一个 Optional 枚举需要存储两部分：

arduino

体验AI代码助手 代码解读 复制代码

```
1 case 标记 + 关联值
```

但 Swift 编译器会做很多布局优化。比如引用类型本身有“空指针”这个无效值，所以：

arduino

体验AI代码助手 代码解读 复制代码

```
1 String?
2 UIViewController?
3 AnyObject?
```

文工程(Context Engineeri...  
6阅读 · 0点赞

ode 版本管理工具 (nvm...  
阅读 · 1点赞

的技术圈子  
加入官方微信群



这类 Optional 通常可以利用指针的 spare bit / nil pointer 表示 `.none`，不一定额外增加存储空间。

例如：

arduino

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 MemoryLayout<Int>.size
2 MemoryLayout<Int?>.size
```

你可能会看到 `Int?` 比 `Int` 更大；但对于某些引用类型 Optional，大小可能和原类型一致。

## Optional 的常见语法，本质都是模式匹配

swift

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 if let value = optional { }
2 guard let value = optional else { }
3 optional ?? defaultValue
4 optional?.method()
5 optional!
```

这些只是不同层次的 unwrap 方式。

比如：

bash

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 if let x = value {
2     print(x)
3 }
```

可以理解成：

swift

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 switch value {
2 case .some(let x):
3     print(x)
4 case .none:
5     break
6 }
```

所以，面试里问 Optional 的实现，重点不是背一句“Optional 是 enum”，而是要说清楚：

Optional 是一个泛型枚举，它通过类型系统表达“值可能不存在”，并由编译器在内存布局和语法层面做了大量优化。

## 2. Task 和 Task.detached 的区别

Swift Concurrency 里，`Task` 是异步执行单元。

但 `Task {}` 和 `Task.detached {}` 差别很大。

## `Task {}`：继承当前上下文

SCSS [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 Task {  
2     await loadData()  
3 }
```

`Task {}` 创建的是一个非结构化任务，但它会继承当前上下文的一些东西，例如：

- 当前 actor 隔离上下文；
- 当前 task priority；
- task-local values；
- 当前取消状态。

官方 Swift Concurrency 文档明确提到，`Task.detached` 创建的新任务默认不继承 actor isolation，也不继承当前任务的 priority 或 task-local state；这反过来也说明普通 `Task` 更接近“延续当前上下文”。

举个例子：

swift [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 @MainActor  
2 final class ViewModel {  
3     var title = ""  
4  
5     func update() {  
6         Task {  
7             // 这里继承 MainActor  
8             title = "loaded"  
9         }  
10    }  
11 }
```

这里的 `Task {}` 继承了 `@MainActor`，所以可以直接访问 `title`。

## `Task.detached {}`：切断上下文

SCSS [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 Task.detached {  
2     await heavyWork()  
3 }
```

`Task.detached` 创建的是一个独立任务。它不继承当前 actor、优先级和 task-local values。

例如：

swift [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 @MainActor
```

```
2 final class ViewModel {
3     var title = ""
4
5     func update() {
6         Task.detached {
7             // 编译错误: 不能直接访问 MainActor 隔离的属性
8             // title = "loaded"
9
10            await MainActor.run {
11                self.title = "loaded"
12            }
13        }
14    }
15 }
```

这就是 `Task.detached` 的关键：它明确脱离当前并发上下文。

## 什么时候用 `Task` ？

大多数业务代码应该优先用 `Task {}`：

▼ ini

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 Task {
2     let result = await api.fetch()
3     self.items = result
4 }
```

尤其是在 `ViewModel`、`View`、`Controller` 里启动异步任务时，`Task {}` 更符合直觉。

## 什么时候用 `Task.detached` ？

`Task.detached` 适合明确不想继承当前上下文的场景：

▼ javascript

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 Task.detached(priority: .background) {
2     await analytics.upload()
3 }
```

或者在 `@MainActor` 环境里做 CPU 密集型计算：

▼ swift

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 @MainActor
2 final class ImageViewModel {
3     func process(image: UIImage) {
4         Task.detached(priority: .userInitiated) {
5             let output = processImage(image)
6
7             await MainActor.run {
8                 self.display(output)
9             }
10        }
11    }
```

```
12 }
```

但要小心：`detached` 不是“更高级的 Task”，它是“更危险的 Task”。  
它会绕开当前 actor 上下文，如果你没处理好取消、优先级和数据隔离，bug 会比较隐蔽。

简单对比

| 特性                     | <code>Task {}</code> | <code>Task.detached {}</code> |
|------------------------|----------------------|-------------------------------|
| 是否继承 actor 上下文         | 是                    | 否                             |
| 是否继承 priority          | 通常是                  | 否，除非显式指定                      |
| 是否继承 task-local values | 是                    | 否                             |
| 是否适合 UI 场景             | 更适合                  | 需要手动切回 MainActor              |
| 使用频率                   | 高                    | 低                             |
| 风险                     | 中                    | 高                             |

一句话：

`Task {}` 是“在当前并发上下文里启动异步任务”；`Task.detached {}` 是“创建一个与当前上下文脱钩的独立任务”。

3. Actor 的实现原理？MainActor 一定在主线程吗？

Actor 是 Swift Concurrency 的核心之一。它解决的是共享可变状态的并发访问问题。

Actor 解决了什么问题？

传统写法里，如果多个线程同时访问同一个对象：

▼ swift

体验AI代码助手

代码解读

复制代码

```
1 final class Counter {
2     var value = 0
3
4     func increment() {
5         value += 1
6     }
7 }
```

多线程同时调用 `increment()`，就可能出现 data race。

Actor 写法：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 actor Counter {
2     private var value = 0
3
4     func increment() {
5         value += 1
6     }
7
8     func getValue() -> Int {
9         value
10    }
11 }
```

Actor 的核心保证是：

同一时间，actor 隔离状态只允许被一个任务访问。

外部访问 actor 方法时通常要 `await`：

▼ csharp

 体验AI代码助手  代码解读 复制代码

```
1 let counter = Counter()
2 await counter.increment()
3 let value = await counter.getValue()
```

这个 `await` 表示：调用可能需要排队，等待进入 actor 的隔离上下文。

## Actor 底层怎么实现？

Swift Evolution 的 Actor 提案里有一个非常关键的实现说明：在实现层面，异步调用 actor 方法会被表示成发送给 actor 的“消息”，这些消息是 partial tasks；每个 actor 实例拥有自己的 serial executor。

可以把 actor 想成：

▼ markdown

 体验AI代码助手  代码解读 复制代码

```
1 actor instance
2 |— isolated state
3 |— serial executor
4   |— job 1
5   |— job 2
6   |— job 3
7
```

外部调用 actor 隔离方法时，不是直接抢锁进去执行，而是把工作投递到 actor 的 executor 上。executor 保证这些访问以串行方式执行。

注意，actor 不是简单等价于“一个线程”：

▼ yaml

 体验AI代码助手  代码解读 复制代码

```
1 Actor != Thread
2 Actor != DispatchQueue
3 Actor 更像是“隔离域 + 串行执行器”
```

Swift runtime 可以在线程池上调度这些任务。actor 保证的是隔离和串行访问，不保证固定运行在某一条线程上。

## Actor 可重入性

Swift actor 默认是 reentrant 的。也就是说，actor 方法执行到 `await` 时，会让出 actor，其他等待中的任务可能进入 actor 执行。

例如：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 actor BankAccount {
2     var balance = 100
3
4     func withdraw(_ amount: Int) async {
5         guard balance >= amount else { return }
6
7         await someNetworkCheck()
8
9         balance -= amount
10    }
11 }
```

这里有一个微妙问题：`await someNetworkCheck()` 之后，`balance` 可能已经被别的任务改了。

所以 actor 能防 data race，但不自动防所有业务逻辑 race。

经验原则是：

不要在 `await` 前后假设 actor 内部状态没有变化。

## MainActor 是什么？

`MainActor` 是一个全局 actor。Swift Evolution 的 Global Actors 提案说明，全局 actor 可以用来表达全局唯一的并发隔离约束，例如“只能在 main thread 或 UI thread 上执行”的代码。

它常用于 UI：

▼ kotlin

 体验AI代码助手  代码解读 复制代码

```
1 @MainActor
2 final class ViewModel: ObservableObject {
3     @Published var title = ""
4
5     func updateTitle() {
6         title = "Hello"
7     }
8 }
```

## MainActor 一定在主线程吗？



工程上可以这样回答：

对 Apple 平台上的 UI 代码来说，`MainActor` 基本就是用来约束主线程执行的；但更精确地说，`MainActor` 是一个全局 actor，它的 executor 与主线程/main dispatch queue 绑定，用于表达主线程隔离。

苹果文档将 `MainActor` 定义为 Swift 的一个全局 actor 类型，常用于主线程相关隔离。

但有个面试里非常容易踩的细节：

`@MainActor` 并不等于任何情况下都会立刻、无条件地帮你切线程。

比如：

▼ SCSS [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 @MainActor
2 func updateUI() {
3     print(Thread.isMainThread)
4 }
```

在 Swift Concurrency 的异步上下文里：

▼ SCSS [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 Task.detached {
2     await updateUI()
3 }
```

这里 `await` 会 hop 到 `MainActor` 。

但是如果你在某些同步、非并发上下文里绕过编译器检查，或者和旧 OC/GCD 代码混用，不能简单把 `@MainActor` 当成 `DispatchQueue.main.async` 的完全替代品。更好的习惯是：

▼ arduino [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 await MainActor.run {
2     // update UI
3 }
```

或者让整个 UI-facing 类型标注：

▼ kotlin [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 @MainActor
2 final class ProfileViewModel {
3     var name = ""
4 }
```

## Actor 小结

Actor 的核心不是线程，而是隔离：

ini

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 Actor = isolated state + serial executor + compiler isolation checking
```

MainActor 是一个特殊的全局 actor，用来表达 UI/main-thread 约束。

它在 Apple UI 编程里通常对应主线程，但你应该从“actor isolation”的角度理解它，而不是把它简单理解成“线程 API”。

## 4. Swift Concurrency 的核心是什么？

Swift Concurrency 的核心不是 `async/await` 语法本身，而是：

用语言级类型系统和运行时协作，提供安全、可组合、可取消、可调度的并发模型。

它主要由几部分组成。

### 1. `async/await`：把异步代码写成顺序代码

以前回调写法：

swift

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 api.fetch { result in
2     switch result {
3     case .success(let data):
4         self.handle(data)
5     case .failure(let error):
6         self.handle(error)
7     }
8 }
```

Swift Concurrency 写法：

csharp

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 let data = try await api.fetch()
2 handle(data)
```

`await` 的本质不是阻塞线程，而是挂起当前任务，让线程去执行别的任务。Swift 官方文档也强调，异步函数在等待时可以暂停并稍后恢复。

### 2. Task：并发执行的基本单位

SCSS

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 Task {
2     await doSomething()
3 }
```

Task 是 Swift 并发运行时调度的单位。它不是线程，多个 Task 可以复用较少数量的线程。

### 3. Structured Concurrency：结构化并发

比如：

▼ csharp

 体验AI代码助手  代码解读 复制代码

```
1 async let user = fetchUser()
2 async let posts = fetchPosts()
3
4 let result = await (user, posts)
```

或者：

▼ csharp

 体验AI代码助手  代码解读 复制代码

```
1 try await withThrowingTaskGroup(of: Image.self) { group in
2     for url in urls {
3         group.addTask {
4             try await downloadImage(url)
5         }
6     }
7
8     var images: [Image] = []
9     for try await image in group {
10         images.append(image)
11     }
12     return images
13 }
```

结构化并发强调：

▼

 体验AI代码助手  代码解读 复制代码

- 1 任务有作用域
- 2 子任务受父任务管理
- 3 错误和取消可以传播
- 4 生命周期清晰

这比随手 `DispatchQueue.global().async` 更安全。

### 4. Actor Isolation：数据竞争安全

Actor 把共享可变状态隔离起来：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 actor Cache {
2     private var storage: [String: Data] = [:]
3
4     func get(_ key: String) -> Data? {
5         storage[key]
6     }
7 }
```

```
8     func set(_ data: Data, for key: String) {
9         storage[key] = data
10    }
11 }
```

访问 actor 隔离状态必须经过 `await`，编译器会帮你挡掉很多 data race。

### 5. Sendable：跨并发域传值的安全约束

`Sendable` 表示一个值可以安全地跨并发边界传递。

▼ rust

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 struct User: Sendable {
2     let id: Int
3     let name: String
4 }
```

Swift 6 之后，严格并发检查变得更重要。很多以前能编译的并发风险代码，在新模式下会变成 warning 或 error。

### Swift Concurrency 的本质

可以这样概括：

▼ csharp

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 async/await 解决异步表达
2 Task 解决执行单元
3 TaskGroup / async let 解决结构化并发
4 Actor 解决共享状态隔离
5 Sendable 解决跨并发域的数据安全
6 MainActor 解决 UI 隔离
```

所以它的核心不是“替代 GCD”，而是：

把并发从库层能力提升为语言层能力，让编译器参与安全检查。

### 5. Swift 与 Objective-C 的主要差异

Swift 和 Objective-C 都能写 Apple 平台应用，但它们的语言哲学完全不同。

#### 1. 静态类型安全 vs 动态消息派发

Objective-C 核心是动态派发：

▼ ini

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 [obj doSomething];
```

底层是消息发送：

▼ less

 体验AI代码助手  代码解读 复制代码

```
1 objc_msgSend(obj, @selector(doSomething));
```

Swift 默认更偏静态：

▼ SCSS

 体验AI代码助手  代码解读 复制代码

```
1 obj.doSomething()
```

编译器能做更多检查和优化。

当然，Swift 也能通过 `@objc`、`dynamic`、`NSObject` 进入 Objective-C runtime，但那不是 Swift 默认模型。

## 2. Optional vs nil

Objective-C：

▼ ini

 体验AI代码助手  代码解读 复制代码

```
1 NSString *name = nil;
```

Swift：

▼ javascript

 体验AI代码助手  代码解读 复制代码

```
1 let name: String? = nil
```

Swift 把空值显式放进类型系统。

这也是 Swift 安全性提升最明显的一点。

## 3. 值类型更重要

Objective-C 主要围绕 class / object。

Swift 里 `struct`、`enum` 是一等公民：

▼ csharp

 体验AI代码助手  代码解读 复制代码

```
1 struct User {
2     let id: Int
3     var name: String
4 }
5
6 enum LoginState {
7     case loggedOut
8     case loggedIn(User)
9 }
10
```

Swift 标准库大量使用值类型：

▼ javascript

 体验AI代码助手  代码解读 复制代码

```
1 String
2 Array
3 Dictionary
4 Set
```

这让 Swift 更强调：

▼ arduino

 体验AI代码助手  代码解读 复制代码

```
1 值语义
2 不可变性
3 copy-on-write
4 线程安全边界
```

## 4. 泛型能力不同

Objective-C 的泛型主要是轻量级泛型，多用于集合标注：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 NSArray<NSString> *names;
```

运行时类型信息并不完整。

Swift 泛型是语言核心能力：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 func max<T: Comparable>(_ a: T, _ b: T) -> T {
2     a > b ? a : b
3 }
```

Swift 的泛型、协议、关联类型、opaque type 可以组合出非常强的抽象能力。

## 5. 协议能力不同

Objective-C protocol 更像接口声明：

▼ objectivec

 体验AI代码助手  代码解读 复制代码

```
1 @protocol Downloader
2 - (void)download;
3 @end
```

Swift protocol 可以有：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 protocol Repository {
2     associatedtype Entity
3
4     func fetch() async throws -> [Entity]
5 }
```

还可以配合 extension 提供默认实现：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 extension Repository {
2     func isEmpty() async throws -> Bool {
3         try await fetch().isEmpty
4     }
5 }
```

Swift 的 protocol 更接近“抽象建模工具”，不只是接口。

## 6. 错误处理不同

Objective-C 常见：

▼ ini

 体验AI代码助手  代码解读 复制代码

```
1 NSError *error = nil;
2 BOOL success = [manager doWork:&error];
```

Swift：

▼ php

 体验AI代码助手  代码解读 复制代码

```
1 do {
2     try manager.doWork()
3 } catch {
4     print(error)
5 }
```

Swift 的 `throw/try/catch` 更接近语言级错误模型。

## 7. 内存管理不同

两者都使用 ARC，但 Swift 更强调所有权和安全访问。

Objective-C 里你会看到：

▼ objectivec

 体验AI代码助手  代码解读 复制代码

```
1 __weak typeof(self) weakSelf = self;
```

Swift 里则是：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 Task { [weak self] in
2     await self?.load()
3 }
```

Swift 还通过 exclusivity enforcement 防止一些同时读写内存的问题。

## 8. 并发模型不同

Objective-C 时代主要靠：

▼ objectivec

 体验AI代码助手  代码解读 复制代码

```
1 dispatch_async
2 NSOperationQueue
3 NSThread
```

Swift 现在有语言级并发：

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 async/await
2 Task
3 TaskGroup
4 actor
5 MainActor
6 Sendable
```

这也是 Swift 和 Objective-C 现代开发体验差异最大的地方之一。

## 9. Runtime 依赖不同

Objective-C 高度依赖 runtime：

▼ csharp

 体验AI代码助手  代码解读 复制代码

```
1 消息发送
2 method swizzling
3 KVC/KVO
4 associated object
5 动态添加方法
```

Swift 默认不依赖这些能力。

Swift 更倾向于编译期静态检查和优化。

但 Swift 与 Objective-C 可以互操作：

▼ less

 体验AI代码助手  代码解读 复制代码

```
1 @objc class MyObject: NSObject {
2     @objc func doSomething() {}
3 }
```



也就是说，Swift 可以进入 OC runtime，但不是所有 Swift 特性都能暴露给 OC，比如：

▼csharp

体验AI代码助手

代码解读

复制代码

```
1 泛型 enum
2 associated value enum
3 actor
4 async 某些形式
5 Swift-only protocol with associatedtype
```

Swift vs OC 总结表

| 维度   | Swift                      | Objective-C             |
|------|----------------------------|-------------------------|
| 类型系统 | 静态、强类型                     | 动态、运行时能力强               |
| 空值   | Optional                   | nil                     |
| 派发   | 默认静态/虚表/协议派发               | objc_msgSend            |
| 主要抽象 | struct、enum、protocol、class | class、protocol、category |
| 泛型   | 强泛型                        | 轻量泛型                    |
| 并发   | async/await、Task、Actor     | GCD、NSOperation         |
| 错误处理 | try/catch                  | NSError                 |
| 安全性  | 编译期更强                      | 运行时更灵活                  |
| 动态能力 | 较弱，需要 @objc/dynamic        | 很强                      |
| 性能优化 | 编译器优化空间大                   | 动态派发成本更明显               |

最后总结

这五个问题可以串成一条线：

▼arduino

体验AI代码助手

代码解读

复制代码

```
1 Optional: Swift 类型安全的基础
2 Task: Swift 异步执行的基本单位
3 Actor: Swift 并发安全的状态隔离机制
4 Swift Concurrency: 语言级并发模型
5 Swift vs OC: 静态安全与动态灵活的取舍
```

如果是面试回答，可以浓缩成这样：

▼typescript

体验AI代码助手

代码解读

复制代码

https://juejin.cn/post/7639078807760011306

第17/19页

- 1 **Optional** 是一个泛型 **enum**，用 ``.some`` 和 ``.none`` 表达值是否存在，并通过类型系统消灭隐式 `nil` 风险。
- 2 ``Task {}`` 会继承当前并发上下文，``Task.detached {}`` 会脱离 `actor`、`priority` 和 `task-local values`。
- 3 **Actor** 通过隔离状态和 `serial executor` 保证同一时间只有一个任务访问其隔离状态；
- 4 **MainActor** 是全局 `actor`，在 **Apple UI** 场景中用于主线程隔离，但应从 `actor isolation` 而不是普通线程 **API** 的角度理解。
- 5 **Swift Concurrency** 的核心是 **async/await**、**Task**、结构化并发、**Actor**、**Sendable** 共同构成的语言级并发安全模型。
- 6 **Swift** 相比 **OC** 更静态、更类型安全、更重视值语义和编译期检查，而 **OC** 更动态、更依赖 `runtime`，灵活但安全边界更靠开发者。

标签： 前端

评论 0



[登录 / 注册](#) 即可发布评论!

暂无评论数据

## 为你推荐

### Swift 新并发框架之 Task

峰之巔 4年前 👁 19k 👍 102 💬 6 iOS Swift

### 【Swift Concurrency】彻底告别回调地狱——async/await、Task、Actor 系统精讲

探索者dx 1月前 👁 389 👍 5 💬 评论 SwiftUI Swift

### GCD vs Swift Concurrency：iOS多线程“老炮”与“鲜肉”的选型大战

SwiftUI大叔 3月前 👁 107 👍 1 💬 评论 iOS

### 深入理解 Swift 6.2 并发：从默认隔离到@concurrent 的完整指南

unravel2025 9月前 👁 1.0k 👍 4 💬 评论 Swift

### Swift 并发全景指南：Thread、Concurrency、Parallelism 一次搞懂

unravel2025 9月前 👁 480 👍 2 💬 评论 Swift

2-9. 【Concurrency】DetachedTask 的上下文继承与 structured concurrency 的区别是什么？

项阿丑 3月前 59 点赞 评论 Swift

Swift Actor 为什么选择可重入设计？——一道让人深思的并发题

探索者dx 1月前 154 点赞 3 评论 Swift SwiftUI

Swift Concurrency 学习笔记

ZHANGYU 3年前 7.8k 点赞 22 评论 Swift

2-30. 【Concurrency】Swift Concurrency 与 OperationQueue 的协作模式有哪些优势与局限？

项阿丑 3月前 39 点赞 评论 Swift

Swift Concurrency从入门到精通

坐南朝北 9月前 552 点赞 4 评论 前端

Swift 6 严格并发检查：@Sendable 与 Actor 隔离的深度解析

山水域 3月前 454 点赞 4 评论 iOS

2-11. 【Concurrency】为什么 Swift Concurrency 强调“结构化”？它如何减少资源泄露和任务孤儿化？

项阿丑 3月前 41 点赞 评论 Swift

Swift Approachable Concurrency(易用并发)

CodingFisher 8月前 305 点赞 评论 iOS Swift SwiftUI

Swift Concurrency：用最简单方式理解苹果官方的异步并发编程

程序员小jobleap 1年前 712 点赞 2 评论 后端 面试 GitHub

Swift Concurrency 中的 Threads 与 Tasks

CodingFisher 8月前 505 点赞 评论



登录掘金领取礼包

更多登录后权益等你解锁

登录 / 注册

AI 助手